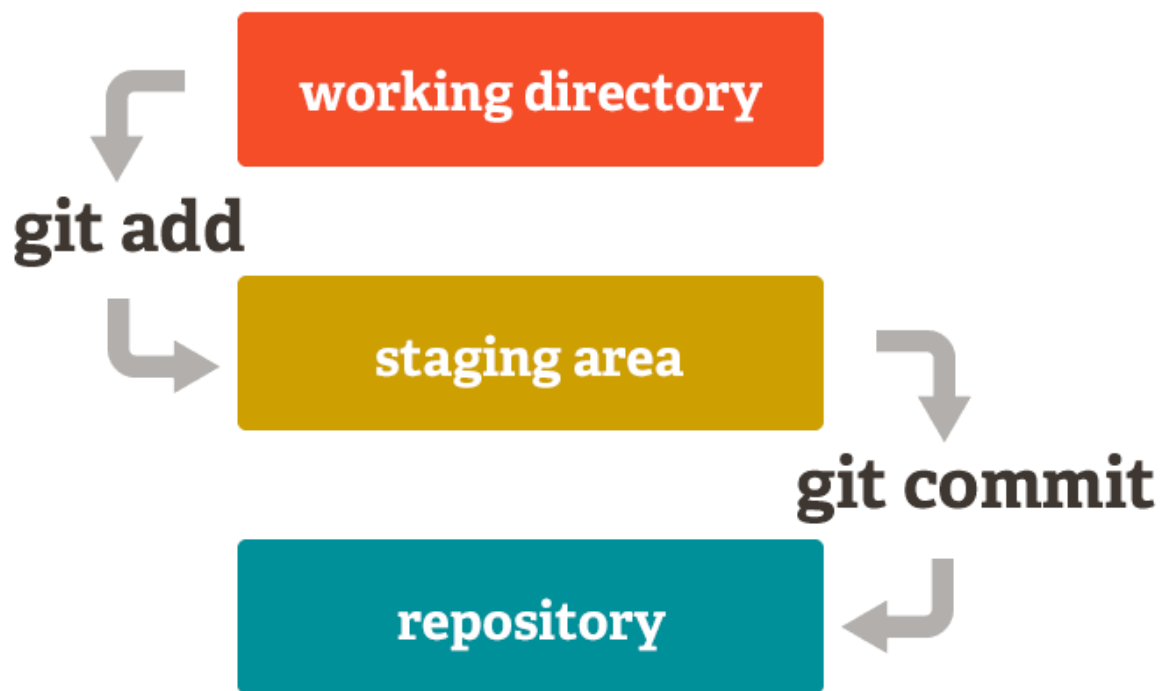




## Phase I: Setting Things Up

I. `git config --global user.name "Your Name"`

- **Explanation:** Tells Git your name. Every time you save your work, Git attaches your name to it so people know who wrote the code.
- II. `git config --global user.email "yourmail@example.com"`
- **Explanation:** Tells Git your email address. This links your local saves to your online GitHub account.
- III. `git init`
- **Explanation:** Initializes (creates) a brand-new, empty Git repository inside your current project folder. This places an invisible tracker inside the folder to start watching your changes.
- IV. `git clone <url>`
- **Explanation:** Copies an existing project from the internet (usually GitHub) directly onto your local machine, complete with all its historical save points.

## Phase II: Saving Your Changes (The Core Flow)

I. `git status`

- **Explanation:** The ultimate check-in command. It tells you exactly what zone your files are currently in—whether they are modified, waiting to be staged, or completely untracked.
- II. git add <filename>
- **Explanation:** Moves a specific file from your **working directory** into the **staging area**. Think of this as putting an item into a cardboard box before sealing it up.
- *Tip:* Running git add . adds absolutely every changed file in your folder all at once.
- III. git commit -m "Your clear message here"
- **Explanation:** Takes everything currently sitting in the staging area and permanently seals it into your local **repository** history with a timestamp and description. This creates a permanent checkpoint in your time machine.

### Phase III: Checking History & Undoing Mistakes

#### I. git log

- **Explanation:** Shows you a timeline of every single commit (checkpoint) made in the project, complete with the author, date, and commit message.
- II. git diff
- **Explanation:** Shows you the exact line-by-line differences between your current edits and your last saved checkpoint. Red lines mean deletions; green lines mean additions.
- III. git restore <filename>
- **Explanation:** Discards any unsaved edits you made to a file in your working directory, instantly resetting it back to how it looked at your last commit. Use this when your experiments break the code.

### Phase IV: Branching (Working without Fear)

#### I. git branch

- **Explanation:** Lists all the branches (parallel universes) in your project. The one with an asterisk next to it is the one you are currently working on.
- II. git branch <branch-name>
- **Explanation:** Creates a new parallel timeline. This lets you experiment with crazy features or risky fixes without touching your safe, working production code.
- III. git checkout <branch-name>
- **Explanation:** Jumps your code workspace over to the specified branch.

- *Tip:* You can use `git checkout -b <branch-name>` to create a new branch and jump straight into it simultaneously.
- IV. `git merge <branch-name>`
- **Explanation:** Combines the changes from the specified branch into your current active branch. Run this from your main branch when your experimental feature is stable and ready to be integrated.

## Connecting Local Git to Cloud GitHub

Once you have your code tracking safely on your machine, you need to push it up to GitHub.

## Phase V: Cloud Operations

I. `git remote add origin <github-repository-url>`

- **Explanation:** Creates a bridge between your local computer project and an empty repository placeholder you created on GitHub. You only need to run this once per project.
- II. `git push -u origin <branch-name>`
- **Explanation:** Uploads your local checkpoints up to GitHub. The `-u` flag remembers your settings so that next time, you can simply type `git push`.
- III. `git pull`
- **Explanation:** The opposite of push. It fetches any updates your teammates uploaded to GitHub and merges them directly down into your local machine workspace so you stay up to date. IV. `git fetch`
- **Explanation:** Downloads the latest history updates from GitHub but *does not* touch or alter your local working code files. It gives you a safe sneak-peek at what others changed before you decide to merge it.

## The Golden Workflow Rules:

1. Make changes to your files.
2. Check your progress with `git status`.
3. Stage the files using `git add ..`
4. Lock in your save point using `git commit -m "Message"`.
5. Upload your work to the cloud using `git push`.

# git THE ESSENTIAL GIT & GITHUB COMMANDS

## CONFIGURATION & SETUP

  
**git config**  
Setting your User Name & Email.

  
**git init**  
Starting a new, local Git repository.

  
**git clone [url]**  
Copying a remote project to your machine.



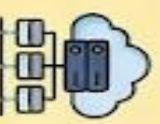
## BRANCHING & MERGING

  
**git branch [name]**  
Creating a new, separate working branch.



  
**git checkout [name]**  
Switching between branches.

  
**git merge [name]**  
Combining changes from one branch into another.



## THE COMMIT WORKFLOW

  
**git status**  
Seeing which files are modified or staged.

  
**git add [file] / .**  
Moving files from Working Dir to Staging Area.

  
**git commit -m "[message]"**  
Saving changes in a 'commit' (snapshot).



## REMOTES & COLLABORATION

  
**git remote add origin [url]**  
Linking your local project to a GitHub repository.

  
**git push origin [branch]**  
Uploading local commits to GitHub.

  
**git pull**  
Downloading remote updates and merging them.

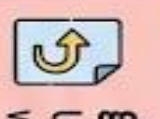


## LOGGING & INSPECTION

  
**git log**  
Viewing a history of commits.

  
**git diff**  
Showing detailed changes between commits or branches.

## RESTORING & UNDOING

  
**git restore [file]**  
Undoing edits in a working file.

  
**git reset [commit]**  
Reverting your repository to an earlier commit.